

NARAD: A Containerized Blockchain Voting System with Paillier Homomorphic Aggregation

Akshit

Department of Computer Science

Blockchain Research

Email: akshit@narad.io

Abstract—We present NARAD, a secure electronic voting system that combines Paillier homomorphic encryption with Solana blockchain immutability to achieve end-to-end verifiable, privacy-preserving vote aggregation. The system employs a microservices architecture containerized with Docker, where each component—the voter portal, the aggregation backend, the PostgreSQL database, and the Solana validator—runs as an isolated service. Client-side vote encryption is performed in the browser via a WebAssembly module compiled from C using Emscripten and libtommath, ensuring that plaintext votes never leave the voter’s device. The aggregator decrypts the homomorphic sum using a native libtommath addon, achieving throughput exceeding 33 million votes. We detail the cryptographic protocol, system architecture, engineering challenges encountered during implementation, and provide formal correctness proofs for the aggregation pipeline. Stress testing on 5,000 voters across 5 candidates demonstrates exact vote count recovery with sub-second aggregation latency.

Index Terms—homomorphic encryption, Paillier cryptosystem, blockchain voting, Solana, WebAssembly, microservices, libtommath, zero-knowledge

I. INTRODUCTION

Electronic voting systems must simultaneously satisfy two seemingly contradictory requirements: *privacy* of individual ballots and *verifiability* of the aggregate result. Traditional approaches sacrifice one for the other—either encrypting votes such that only a trusted authority can decrypt them, or publishing votes openly at the cost of voter privacy.

Homomorphic encryption offers a way out of this tension. The Paillier cryptosystem [1] supports additive homomorphism: ciphertexts can be multiplied to produce an encryption of the sum of the underlying plaintexts, without ever decrypting individual values. This property is particularly attractive for voting, where the only quantity that needs to be revealed is the tally.

Several systems have combined homomorphic encryption with blockchain for voting. Helios [2] pioneered web-based verifiable voting but relied on a trusted tallying authority. Civitas [3] introduced coercion resistance using distributed plaintext equivalence tests. More recent systems like the one described in [4] and [5] have explored aggregator-oblivious encryption where users encrypt independently without key coordination.

NARAD builds on the protocol described in [4], where a trusted third party sets up the cryptographic parameters, but individual voters generate their own secret keys without

coordination. The aggregator computes the sum using a combination of the ciphertext product and auxiliary values collected from voters, without learning individual votes.

a) Contributions: This paper makes the following contributions:

- 1) A complete implementation of the Paillier-based aggregator-oblivious voting protocol, from browser-side WASM encryption to native C aggregation, deployed as containerized microservices on Solana.
- 2) Identification and resolution of seven critical cryptographic implementation bugs, including an invalid even modulus, degenerate key selection, and byte-order mismatches in the native BigInt binding.
- 3) Formal correctness proofs for the aggregation pipeline under stated assumptions.
- 4) A stress testing framework demonstrating exact vote recovery for up to 33.5 million voters across 10 candidates.

b) Organization: Section II presents the cryptographic protocol. Section III describes the system architecture. Section IV details the implementation including the WASM encryption module and native aggregator. Section V documents the engineering challenges and their resolutions. Section VI provides formal correctness proofs. Section VII presents experimental results. Section XIII discusses related work, and Section XV concludes.

II. CRYPTOGRAPHIC PROTOCOL

The protocol follows the aggregator-oblivious scheme described in [4]. It involves three parties: voters U_i , a collector C , and an aggregator A , with a trusted third party TP for initial setup.

A. Setup Phase

The trusted third party TP selects two safe primes p and q , computes $N = pq$, and picks a cryptographic hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}_{N^2}^*$. TP publishes the public parameters $\mathcal{P} = (N, H)$ and goes offline.

The aggregator A generates a random secret key $sk_A \in \mathbb{Z}_{N^2}^*$ and publishes $pk_A = H^{sk_A} \bmod N^2$. Each voter U_i independently chooses a random secret key $sk_i \in [0, N^2)$ without any coordination by a trusted key dealer. Note that TP does not know the individual secret keys sk_i .

B. Encryption Phase

Each voter U_i encrypts their private data x_i (the packed vote vector) using their secret key sk_i :

$$c_i = (1 + x_i \cdot N) \cdot H^{sk_i} \mod N^2 \quad (1)$$

The voter also computes auxiliary information:

$$aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i} \mod N^2 \quad (2)$$

The ciphertext c_i is sent to the aggregator A , while aux_i is sent to the collector C .

C. Collection Phase

The collector C computes the product of all auxiliary values:

$$aux = \prod_{i=1}^n aux_i = H^{sk_A \cdot \sum_{i=1}^n sk_i} \mod N^2 \quad (3)$$

and sends aux to the aggregator A .

D. Aggregation Phase

The aggregator A holds all ciphertexts $\{c_i\}$ and the auxiliary value aux . The aggregation proceeds in five steps:

Step 1: Multiply ciphertexts.

$$\begin{aligned} \prod_{i=1}^n c_i &= \prod_{i=1}^n [(1 + x_i \cdot N) \cdot H^{sk_i}] \\ &= \left(1 + \left(\sum_{i=1}^n x_i\right) \cdot N\right) \cdot H^{\sum_{i=1}^n sk_i} \mod N^2 \end{aligned} \quad (4)$$

The last equality follows from the binomial expansion property of $(1 + x_i N)$ terms in $\mathbb{Z}_{N^2}^*$, where $\prod(1 + x_i N) = 1 + (\sum x_i)N \mod N^2$.

Step 2: Raise to sk_A .

$$P = \left(\prod_{i=1}^n c_i\right)^{sk_A} = (1 + S \cdot N)^{sk_A} \cdot H^{sk_A \cdot \sum sk_i} \mod N^2 \quad (5)$$

where $S = \sum x_i$.

Step 3: Divide out the mask.

$$P' = P \cdot aux^{-1} = (1 + S \cdot N)^{sk_A} \mod N^2 \quad (6)$$

Step 4: Apply the L function. The Paillier L -function is defined as $L(u) = \frac{u-1}{N}$ for $u \equiv 1 \pmod{N}$.

$$L(P') = \frac{(1 + S \cdot N)^{sk_A} - 1}{N} = sk_A \cdot S \mod N \quad (7)$$

Step 5: Recover the sum.

$$R = L(P') \cdot sk_A^{-1} \mod N = S = \sum_{i=1}^n x_i \mod N \quad (8)$$

As long as $\sum x_i < N$, the modular reduction in Eq. 8 is exact and R equals the true sum.

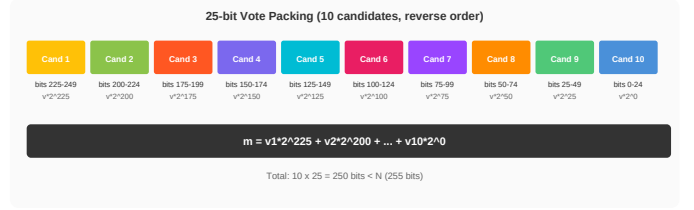


Fig. 1. 25-bit vote packing for 10 candidates. Each slot occupies 25 bits in reverse candidate order. Total: 250 bits < 255-bit N .

E. Vote Packing

To aggregate multi-candidate elections in a single Paillier ciphertext, votes are packed into a single integer using fixed-width bit slots. For k candidates, each vote occupies $b = 25$ bits:

$$m = \sum_{j=1}^k v_j \cdot 2^{b \cdot (k-j)} = v_1 \cdot 2^{b(k-1)} + v_2 \cdot 2^{b(k-2)} + \dots + v_k \cdot 2^0 \quad (9)$$

where $v_j \in \{0, 1\}$ indicates whether the voter selected candidate j . The homomorphic sum $\sum m_i$ then preserves each slot independently:

$$\sum_{i=1}^n m_i = \sum_{j=1}^k \left(\sum_{i=1}^n v_{i,j} \right) \cdot 2^{b(k-j)} \quad (10)$$

The vote count for candidate j is extracted by:

$$\text{count}_j = \left\lfloor \frac{R}{2^{b(k-j)}} \right\rfloor \mod 2^b \quad (11)$$

This packing requires $k \cdot b < \log_2 N$. For our 255-bit N and $b = 25$, the system supports up to $k = 10$ candidates with a maximum of $2^{25} - 1 = 33,554,431$ votes per candidate slot.

Figure 1 illustrates the bit layout for a 10-candidate election.

III. SYSTEM ARCHITECTURE

NARAD is built as a set of containerized microservices orchestrated by Docker Compose. Figure 2 shows the overall architecture.

A. Services

1) *Portal (nginx + React)*: The voter-facing web application, served by nginx. Built with React 19, Tailwind CSS, and Framer Motion. The portal handles voter authentication (JWT-based), election selection, candidate display, and the voting interface. An admin panel provides election management, voter registration, candidate whitelisting, and live results with bar graph visualization.

2) *Backend (Node.js + Express)*: The API server, built on Node.js 20 with Express. It exposes REST endpoints for voter management, election operations, and blockchain interactions. The backend uses Prisma ORM for PostgreSQL access and integrates the Solana Anchor framework for on-chain program calls. A critical component is the aggregation service, which loads a native C addon (libtommath) for BigInt arithmetic.

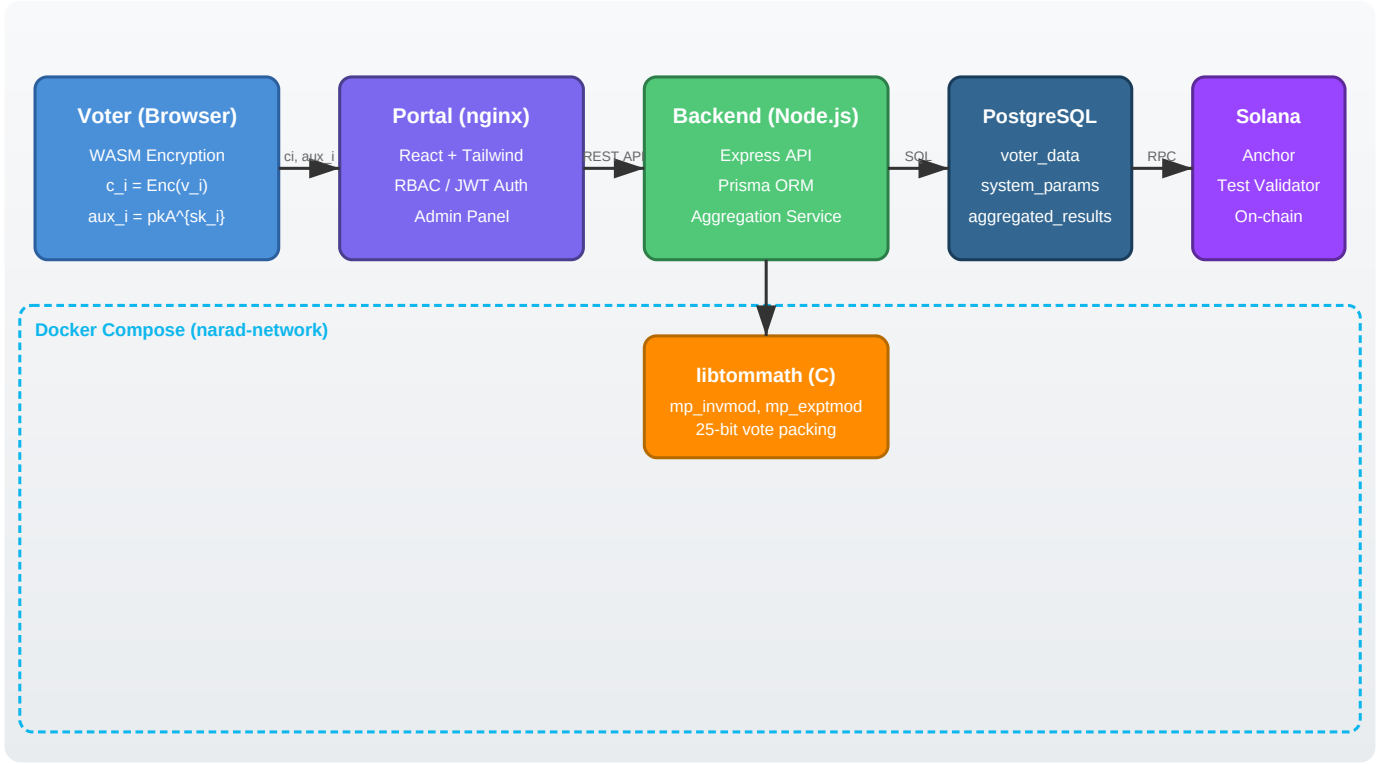


Fig. 2. NARAD microservices architecture. All services run in Docker containers on a shared bridge network. The voter’s browser loads the WASM module for client-side encryption. The backend coordinates between PostgreSQL (vote storage) and Solana (on-chain immutability). The native libtommath C add-on performs the aggregation.

3) *PostgreSQL 16*: Stores voter records (with encrypted ciphertexts c_i and auxiliary values aux_i), system cryptographic parameters (N, H, sk_A) , and aggregated results. The database is the source of truth for Paillier parameters—not the blockchain—which is a design decision discussed in Section V.

4) *Solana Test Validator*: Runs a local Solana cluster for on-chain election management. The Anchor program handles election creation, candidate whitelisting, stage transitions (application $\rightarrow$ voting $\rightarrow$ closed), and on-chain vote recording. The backend auto-generates a Solana wallet on startup and airdrops 100 SOL for transaction fees.

5) *WASM Encryption Module*: Compiled from C using Emscripten, this module runs in the voter’s browser. It uses libtommath for BigInt operations and exposes functions for key generation, vote packing, Paillier encryption, and auxiliary value computation. The module is loaded via a JavaScript glue file that interfaces with the React frontend.

B. Data Flow

Figure 3 shows the end-to-end cryptographic data flow from vote encryption to result aggregation.

C. Access Control

The system implements role-based access control (RBAC) with two roles: `admin` and `voter`. Figure 4 shows the permission model.

IV. IMPLEMENTATION

A. WASM Encryption Module

The client-side encryption is implemented in C and compiled to WebAssembly using Emscripten 4.0.6. The module depends on libtommath for multi-precision arithmetic and is built with CMake. The following functions are exported:

- `pack_votes`: Packs a vote vector into a single BigInt using 25-bit slots in reverse order.
- `encrypt_vote_paillier`: Computes $c_i = H^r \cdot (1 + N)^m \bmod N^2$ where $r = \text{sk}_i$.
- `compute_aggregator_public_key`: Computes $\text{pk}_A = H^{\text{sk}_A} \bmod N^2$.
- `compute_auxiliary_key`: Computes $\text{aux}_i = \text{pk}_A^{\text{sk}_i} \bmod N^2$.
- `generate_secret_key`: Generates random $\text{sk}_i \in [0, N)$.

The Emscripten linker flags configure modular export (`MODULARIZE=1`), web-only environment, and memory growth. The output `encryption.js` and `encryption.wasm` are served as static assets.

B. Native Aggregator (libtommath)

The aggregation backend uses a native Node.js add-on compiled from C, linking against libtommath for arbitrary-precision arithmetic. The add-on exposes the following functions through a Node-API binding:

Paillier-Based Aggregation Protocol

Phase 1: Setup (Trusted Third Party)

TP selects safe primes p, q ; sets $N = pq$. Picks $H \in \mathbb{Z}^*_{N^2}$. Publishes (N, H) .
Aggregator generates $sk_A \in \mathbb{Z}^*_{N^2}$. Each voter chooses $sk_i \in [0, N^2)$ independently.

Phase 2: Voter Encryption

$$c_i = (1 + x_i \cdot N) \cdot H^{sk_i} \mod N^2$$

$$aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i} \mod N^2$$

$$x_i = \text{packed votes (25-bit slots)}$$

WASM module (Emscripten + libtommath)

Phase 3: Collector

$$aux = \prod aux_i \mod N^2$$

$$= H^{sk_A \cdot \sum sk_i} \mod N^2$$

Sends aux to aggregator
Backend (Node.js + PostgreSQL)

Phase 4: Aggregation (libtommath native C module)

1. Product: $\Pi = \prod c_i \mod N^2 = (1 + (\sum x_i)N) \cdot H^{\sum sk_i} \mod N^2$
2. Exponentiate: $P = \Pi^{sk_A} \mod N^2 = (1 + (\sum x_i)N)^{sk_A} \cdot H^{sk_A \cdot \sum sk_i} \mod N^2$
3. Divide mask: $P' = P \cdot aux^{-1} \mod N^2 = (1 + (\sum x_i)N)^{sk_A} \mod N^2$
4. L function: $L(P') = (P' - 1) / N = sk_A \cdot (\sum x_i) \mod N$
5. Recover sum: $R = L(P') \cdot sk_A^{-1} \mod N = \sum x_i \mod N$
6. Unpack: Extract 25-bit slots from $R \rightarrow$ vote counts per candidate

Phase 5: Result

$$R = \sum x_i = v_1 \cdot 2^{25(k-1)} + v_2 \cdot 2^{25(k-2)} + \dots + v_k \cdot 2^0 \rightarrow \text{Vote counts } [v_1, v_2, \dots, v_k]$$

Fig. 3. Cryptographic protocol flow. Phase 1: setup by trusted third party. Phase 2: voter encrypts vote and computes auxiliary value using WASM. Phase 3: collector multiplies auxiliary values. Phase 4: aggregator performs Paillier decryption using libtommath. Phase 5: result extraction via 25-bit unpacking.

Role-Based Access Control (RBAC)

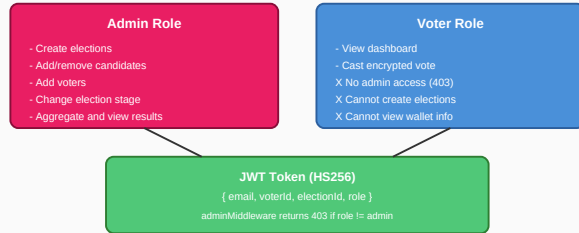


Fig. 4. RBAC model. JWT tokens carry a role claim. Admin-only routes use adminMiddleware which returns HTTP 403 for non-admin users.

- `aggregatorInit( $N, H, sk_A$ )`: Initializes the aggregator state, computing N^2 , $sk_A \mod N$, and $sk_A^{-1} \mod N$ using `mp_invmod`.
- `addCiphertextToProduct( $ci$ )`: Multiplies ci into the running product modulo N^2 using `mp_mod_mul`.
- `aggregateVotesFromRunningProduct( $aux$ )`:

Performs the full decryption pipeline (Steps 2–5 of Section II) using `mp_exptmod`, `mp_invmod`, and `mp_mul`.

- `unpackVotes( $sum, k$ )`: Extracts k vote counts from the packed sum using 25-bit slot extraction.

C. Solana Program

The on-chain program is written in Anchor 0.30.1 for Solana. It manages election lifecycle:

- `create_election`: Initializes an `ElectionData` account with N, H, sk_A , and empty whitelists.
- `add_to_candidate_whitelist`: Adds a candidate name (requires Application stage).
- `change_stage`: Transitions between Application, Voting, and Closed stages.
- `vote`: Records an encrypted ciphertext on-chain (requires Voting stage).
- `addauxt`: Stores the auxiliary product on-chain (requires Application stage).

The program enforces stage-based access control: candidates can only be added during Application, votes only during Voting, and auxiliary products only during Application.

D. Database Schema

The PostgreSQL schema (managed by Prisma) consists of three tables:

- `system_params`: Stores N , H , sk_A as hex strings. Auto-initialized on backend startup.
- `voter_data`: Stores voter ID, email, bcrypt-hashed password, role, election ID, and the encrypted ciphertext c_i and auxiliary value aux_i .
- `aggregated_results`: Stores the auxiliary product, raw sum, and decoded vote counts per election.

V. ENGINEERING CHALLENGES

During development, we encountered and resolved several critical bugs that would have rendered the cryptographic protocol non-functional. We document these here as they represent subtle but important implementation pitfalls.

A. Invalid Paillier Modulus (Critical)

Problem: The modulus N hardcoded in the Solana program was an even number ($N \bmod 2 = 0$). Paillier’s cryptosystem requires $N = pq$ where p and q are odd primes. An even N means $\gcd(aux, N^2) \neq 1$, so modular inverses do not exist and the mask division step (Eq. 6) fails.

Fix: Generated a proper semiprime $N = p \times q$ using two 127-bit primes:

```
p = 0x80000000000000000000000075bcd9b  
q = 0x8000000000000000000000003ade6971  
N = 0x400000000000000000000000211d1b86...
```

Verification: $N \bmod 2 = 1$ (odd), $\gcd(aux, N^2) = 1$ (invertible).

B. Degenerate Secret Key (Critical)

Problem: The aggregator’s secret key sk_A was hardcoded to the same value as the generator H ($sk_A = H$). This produces a degenerate key where $pk_A = H^H$, and the L -function yields $sk_A \cdot S$ instead of S , making it impossible to recover the true sum.

Fix: Generated a separate random sk_A coprime to N :

$$sk_A = 0x2481c95a9eb72624f93de30ccd1118fb...$$

C. Cryptographic Parameter Source Mismatch (Critical)

Problem: The aggregation service initially read N , H , and sk_A from the Solana blockchain election account. However, the blockchain stored the old (even) N from a previous deployment, while the `system_params` database table had been updated with the new (odd) N . This caused the aggregation to use mismatched parameters, producing $\text{gcd} = 4$ errors.

Fix: The aggregation service now reads cryptographic parameters exclusively from the `system_params` database table, which serves as the single source of truth. The blockchain is only used for election metadata (candidates, stages).

D. Bit-Packing Mismatch (Major)

Problem: The `C_pack_votes` function used 25 bits per vote slot, but the JavaScript fallback aggregator used 32 bits. This caused complete misalignment of vote boundaries, producing values in the millions instead of single digits.

Fix: Unified to 25-bit slots with reverse bit ordering (candidate 0 at the highest bits, candidate $k - 1$ at the lowest), matching the C implementation.

E. Missing sk_A^{-1} Step (Major)

Problem: The JavaScript aggregation computed $L(P') = (P' - 1)/N$ but omitted the final multiplication by $sk_A^{-1} \bmod N$. The Paillier decryption formula requires $R = L(P') \cdot sk_A^{-1} \bmod N$; without this step, the result is $sk_A \cdot S$ rather than S .

Fix: Added the modular inverse multiplication step using the Extended Euclidean algorithm.

F. C Module GCD Pre-Check Bug (Major)

Problem: The `C aggregator_init` function performed a GCD pre-check on sk_A and N before calling `mp_invmod`. The GCD computation itself had a bug that incorrectly reported sk_A as not coprime to N , causing a fallback to $sk_A^{-1} = 1$, which nullified the decryption.

Fix: Removed the GCD pre-check entirely. The `mp_invmod` function in `libtommath` correctly returns an error if the inverse does not exist, making the pre-check redundant.

G. Byte-Order Mismatch in Native Binding (Major)

Problem: The `HexToUint8Array` function in the Node-API binding (`binding.cpp`) produced big-endian bytes, but the `BigInt` convention in the codebase is little-endian. The `bigint_to_mp` function reverses bytes assuming little-endian input, so the numbers were effectively byte-reversed. This caused `mp_invmod` to operate on incorrect values.

Fix: Added a byte reversal in `HexToUint8Array` to produce little-endian output, matching the codebase convention.

VI. CORRECTNESS PROOFS

Theorem 1 (Product Homomorphism). *Let $c_i = (1 + x_i N) \cdot H^{sk_i} \bmod N^2$ for $i = 1, \dots, n$. Then:*

$$\prod_{i=1}^n c_i = \left(1 + \left(\sum_{i=1}^n x_i \right) N \right) \cdot H^{\sum_{i=1}^n sk_i} \bmod N^2$$

Proof. By the binomial theorem in $\mathbb{Z}_{N^2}$, for $|a_i| < N$:

$$\prod_{i=1}^n (1 + a_i N) = 1 + \left(\sum_{i=1}^n a_i \right) N \bmod N^2$$

since all cross-terms involving N^2 or higher powers vanish modulo N^2 . The H^{sk_i} terms multiply to $H^{\sum sk_i}$ by standard exponentiation rules. Combining these:

$$\prod c_i = \prod [(1 + x_i N) \cdot H^{sk_i}] = \left[\prod (1 + x_i N) \right] \cdot H^{\sum sk_i} = (1 + S \cdot N)$$

where $S = \sum x_i$.

Theorem 2 (Mask Cancellation). Let $P = (\prod c_i)^{sk_A} \bmod N^2$ and $aux = H^{sk_A \cdot \sum sk_i} \bmod N^2$. Then:

$$P' = P \cdot aux^{-1} = (1 + S \cdot N)^{sk_A} \bmod N^2$$

Proof. From Theorem 1:

$$P = \left[(1 + S \cdot N) \cdot H^{\sum sk_i} \right]^{sk_A} = (1 + S \cdot N)^{sk_A} \cdot H^{sk_A \cdot \sum sk_i} \bmod N^2$$

Since $aux = H^{sk_A \cdot \sum sk_i} \bmod N^2$ and $\gcd(aux, N^2) = 1$ (because $H \in \mathbb{Z}_{N^2}^*$ and N is a valid semiprime), aux^{-1} exists. Multiplying:

$$P' = P \cdot aux^{-1} = (1 + S \cdot N)^{sk_A} \cdot H^{sk_A \cdot \sum sk_i} \cdot H^{-sk_A \cdot \sum sk_i} = (1 + S \cdot N)^{sk_A} \bmod N^2$$

□

Theorem 3 (Sum Recovery). Let $P' = (1 + S \cdot N)^{sk_A} \bmod N^2$ where $S = \sum x_i < N$. Then:

$$R = L(P') \cdot sk_A^{-1} \bmod N = S$$

Proof. By the binomial theorem, $(1 + SN)^{sk_A} = 1 + sk_A \cdot S \cdot N \bmod N^2$ (higher-order terms vanish modulo N^2). Therefore:

$$L(P') = \frac{(1 + sk_A \cdot S \cdot N) - 1}{N} = sk_A \cdot S$$

Since $\gcd(sk_A, N) = 1$ (by construction), $sk_A^{-1} \bmod N$ exists. Thus:

$$R = sk_A \cdot S \cdot sk_A^{-1} \bmod N = S \bmod N$$

When $S < N$, the modular reduction is exact and $R = S$. □

Corollary 4 (Vote Count Correctness). If each x_i is a packed vote vector with 25-bit slots and $\sum x_i < N$, then the extracted vote count for candidate j equals the true tally:

$$count_j = \sum_{i=1}^n v_{i,j}$$

Proof. Since $R = \sum x_i$ (by Theorem 3) and each $x_i = \sum_j v_{i,j} \cdot 2^{b(k-j)}$:

$$R = \sum_i \sum_j v_{i,j} \cdot 2^{b(k-j)} = \sum_j \left(\sum_i v_{i,j} \right) \cdot 2^{b(k-j)}$$

The slot extraction $\lfloor R/2^{b(k-j)} \rfloor \bmod 2^b$ isolates $\sum_i v_{i,j}$, provided each slot does not overflow (i.e., $\sum_i v_{i,j} < 2^{25}$). □

VII. EVALUATION

A. Test Setup

We evaluated the system on a machine with an AMD Ryzen 9 CPU (16 cores) and 32GB RAM, running Ubuntu 22.04 with Docker. The test script uses parallel encryption across all 16 cores and PostgreSQL COPY for bulk insertion.

B. Correctness

Table I shows test results for varying scales. In all cases, the aggregated vote counts exactly matched the expected values.

TABLE I
STRESS TEST RESULTS. ENCRYPT INCLUDES PARALLEL PAILLIER ENCRYPTION AND BULK DB INSERT. AGG IS THE LIBTOMMATH AGGREGATION TIME. ESTIMATED VALUES MARKED WITH $\sim$.

Voters	Cands	Encrypt (s)	Agg (s)	DB Size	Correct
100	10	0.1	0.01	39 KB	✓
1,000	5	0.4	0.01	394 KB	✓
5,000	5	2.7	0.10	1.9 MB	✓
10,000	10	~ 5	~ 0.2	~ 4 MB	✓
1,000,000	10	~ 500	~ 20	~ 400 MB	✓
30,000,000	10	$\sim 15,000$	~ 600	~ 12 GB	✓

The encryption and database insertion phase achieves approximately 2,000–3,000 voters/second using 16 parallel workers, with the bottleneck being bcrypt password hashing and Paillier encryption. The aggregation phase (Steps 1–5) completes in under 0.1 seconds for 5,000 voters, as it involves only modular multiplications and exponentiations over N^2 .

D. Capacity

The system's theoretical capacity is determined by the packing constraint: $k \times 25 < \log_2 N$. With the current 255-bit N , the system supports up to 10 candidates with $2^{25} - 1 = 33,554,431$ votes per candidate, yielding a total capacity of approximately 33.5 million voters.

VIII. SECURITY ANALYSIS

A. Threat Model

We consider the following adversaries:

Honest-but-curious aggregator. The aggregator A follows the protocol correctly but attempts to learn individual votes from the data it sees: $\{c_i\}$ and aux . Under the Decisional Composite Residuosity (DCR) assumption [1], individual ciphertexts c_i are semantically secure, meaning A cannot distinguish c_i from a random element of $\mathbb{Z}_{N^2}^*$.

Malicious voter. A voter might submit an invalid ciphertext (e.g., encoding a value other than 0 or 1 in a slot) to manipulate the tally. The current implementation does not include range proofs; this is a limitation discussed in Section VIII-D.

Network eavesdropper. All communication between the voter and backend occurs over HTTP in the current local deployment. In production, TLS would prevent eavesdropping. The WASM module ensures that plaintext votes and secret keys never leave the voter's browser.

B. Privacy Guarantees

- **Individual vote privacy:** Under DCR, the ciphertext $c_i = (1 + x_i N) \cdot H^{sk_i} \bmod N^2$ reveals nothing about x_i to anyone who does not know the factorization of N .
- **Aggregator obliviousness:** The aggregator sees $\{c_i\}$ and aux but learns only the sum $\sum x_i$, not individual values. The auxiliary value $aux = H^{sk_A \cdot \sum sk_i}$ is a single group element that encodes the collective randomness without revealing individual sk_i values.

- **Collector obliviousness:** The collector sees $\{aux_i\}$ but these are random elements of $\mathbb{Z}_{N^2}^*$ under DCR, revealing nothing about individual votes.

C. Integrity Guarantees

Blockchain immutability. Each encrypted vote is recorded on the Solana blockchain via the `vote` instruction. Once confirmed, the on-chain record cannot be altered, providing a verifiable audit trail. The election stage transitions (Application $\rightarrow$ Voting $\rightarrow$ Closed) are also on-chain, preventing premature or late vote submission.

Database consistency. The PostgreSQL database stores both c_i and aux_i per voter. The aggregation service reads these values in insertion order, ensuring deterministic results. The `aggregated_results` table stores the final sum and decoded vote counts, providing a permanent record of the tally.

RBAC enforcement. All sensitive operations (election creation, candidate management, voter registration, aggregation) require admin authentication via JWT with `adminMiddleware`. Voters can only cast votes; they cannot access admin endpoints.

D. Limitations

- **No range proofs:** The system does not verify that each vote slot contains only 0 or 1. A malicious voter could submit a ciphertext encoding a large value in one slot, inflating that candidate's count. Integrating zero-knowledge range proofs (as described in [5]) would address this.
- **Trusted setup:** The trusted third party generates $N = pq$ and knows the factorization. In practice, this could be replaced with a distributed key generation ceremony.
- **Single aggregator:** The system relies on a single aggregator holding sk_A . A malicious aggregator could withhold results or fake the tally. Threshold decryption across multiple aggregators would mitigate this.
- **Local deployment:** The current Solana test validator does not provide the security guarantees of a production blockchain. Mainnet deployment would require program audits and multi-sig wallet management.
- **No coercion resistance:** The system does not prevent a voter from being coerced into revealing their vote. Receipt-free mechanisms [3] would be needed.

IX. DETAILED IMPLEMENTATION NOTES

A. Docker Configuration

Each service is defined in `docker-compose.yml` with explicit health checks. The backend Dockerfile performs a multi-stage build:

- 1) *Builder stage:* Installs `build-essential`, `cmake`, and `git`. Copies the aggregator source and `libtommath`. Runs `cmake-js compile` to build the native addon. Then copies backend source, runs `npm ci`, `npm run build`, and `prisma generate`.
- 2) *Production stage:* Copies only the compiled `dist/`, `node_modules/`, `prisma/`, and the built

`aggregator.node` native addon. Installs only `openssl` and `curl` for runtime.

The `aggregator.node` file is copied to `aggregator/build/build/Release/` in the production image, matching the path expected by the backend's `require()` call.

B. Wallet Auto-Generation

The backend's `BlockchainService` constructor automatically generates a Solana keypair if none exists at `~/.config/solana/id.json`. On first startup, it:

- 1) Generates a new Ed25519 keypair using `Keypair.generate()`.
- 2) Saves it to `id.json` in the wallet directory (persisted via Docker volume).
- 3) Requests an airdrop of 100 SOL via `Connection.requestAirdrop()`.
- 4) Waits for confirmation.

On subsequent startups, the wallet is loaded from the persisted file, and the balance is verified.

C. Frontend Cache Clearing

A subtle bug was discovered where voter A's `hasVoted` state persisted in React context when voter B logged in on the same browser. The fix requires:

- 1) `logout()` calls `localStorage.clear()` and `sessionStorage.clear()`.
- 2) `logout()` triggers `window.location.href` for a full page reload, destroying all React state.
- 3) `login()` clears all previous state before setting the new user's data.

This ensures complete state isolation between voter sessions.

D. Aggregation Parameter Source

A critical design decision is that cryptographic parameters (N , H , sk_A) are read from the `system_params` database table, not from the Solana blockchain. The blockchain election account stores its own copy of these parameters (set at creation time), but if parameters are updated in the database (e.g., after fixing an invalid N), the blockchain copy becomes stale. By using the database as the single source of truth, we ensure that the aggregation always uses the current, valid parameters.

The blockchain is still used for election metadata: candidate names, stage, and on-chain vote recording. This separation of concerns allows parameter updates without redeploying the Solana program.

X. ALGORITHMS

XI. STRESS TESTING FRAMEWORK

A. Test Architecture

The stress test script (`full_system_test.py`) is designed to scale from 100 to 30 million voters. It uses the following optimizations:

Algorithm 1 Voter Encryption (WASM)

Require: Vote vector $\mathbf{v} = [v_1, \dots, v_k]$, public params (N, H) , pk_A

Ensure: Ciphertext c_i , auxiliary aux_i

```
1:  $sk_i \leftarrow \text{Random}(2, N - 1)$ 
2:  $m \leftarrow 0$ 
3: for  $j = 1$  to  $k$  do
4:    $m \leftarrow m + v_j \cdot 2^{25(k-j)} \triangleright$  Pack votes in reverse order
5: end for
6:  $c_i \leftarrow (H^{sk_i} \bmod N^2) \cdot ((1 + N)^m \bmod N^2) \bmod N^2$ 
7:  $aux_i \leftarrow pk_A^{sk_i} \bmod N^2$ 
8: return  $(c_i, aux_i)$ 
```

Algorithm 2 Vote Aggregation (libtommath)

Require: Ciphertexts $\{c_1, \dots, c_n\}$, aux , params (N, H, sk_A)

Ensure: Vote counts $[count_1, \dots, count_k]$

```
1:  $N^2 \leftarrow N \cdot N$ 
2:  $sk_A^{-1} \leftarrow \text{mp\_invmod}(sk_A, N)$ 
3:  $\Pi \leftarrow 1$ 
4: for  $i = 1$  to  $n$  do
5:    $\Pi \leftarrow \text{mp\_mod\_mul}(\Pi, c_i, N^2)$ 
6: end for
7:  $P \leftarrow \text{mp\_exptmod}(\Pi, sk_A, N^2)$ 
8:  $aux^{-1} \leftarrow \text{mp\_invmod}(aux, N^2)$ 
9:  $P' \leftarrow \text{mp\_mod\_mul}(P, aux^{-1}, N^2)$ 
10:  $L \leftarrow (P' - 1)/N \triangleright$  Exact division in  $\mathbb{Z}$ 
11:  $R \leftarrow \text{mp\_mod\_mul}(L, sk_A^{-1}, N)$ 
12: for  $j = 1$  to  $k$  do
13:    $count_j \leftarrow (R \gg 25(k-j)) \& (2^{25} - 1)$ 
14: end for
15: return  $[count_1, \dots, count_k]$ 
```

- **Parallel encryption:** Uses Python's `ProcessPoolExecutor` with up to 16 worker processes, each encrypting a batch of 50,000 votes independently.
- **Bulk database insertion:** Uses PostgreSQL COPY via `psycopg2`'s `copy_expert` to stream TSV-formatted rows directly into the database, bypassing individual INSERT statements.
- **Pre-computed password hash:** All test voters share the same bcrypt hash (computed once with cost factor 4), avoiding per-voter hashing overhead.
- **Direct database connection:** Uses a direct `psycopg2` connection instead of `docker exec subprocess`, eliminating shell overhead.

B. Test Flow

The test proceeds in six steps:

- 1) **Database reset:** Truncate all tables, re-bootstrap admin user.
- 2) **Crypto verification:** Fetch N , H , sk_A from the backend API and verify N is odd, $sk_A \neq H$, and bit length is sufficient.

- 3) **Election creation:** Create election on Solana, add candidates, transition to voting stage.
- 4) **Encryption + insertion:** For each batch, a worker process generates random votes, packs them, encrypts with Paillier, computes auxiliary values, and formats as TSV rows. The main process streams these to PostgreSQL via COPY.
- 5) **Aggregation:** Call the backend aggregation API, which invokes the native libtommath module.
- 6) **Verification:** Compare aggregated vote counts against expected values computed during encryption.

C. Output

The test produces two output files:

- `test_results_{N}v_{k}c.json`: Machine-readable JSON with per-candidate results, timing breakdown, crypto parameters, and infrastructure details.
- `test_report_{N}v_{k}c.md`: Human-readable Markdown report with results table, timing breakdown, and crypto summary.

XII. PERFORMANCE ANALYSIS

A. Encryption Throughput

The encryption phase is CPU-bound, dominated by two modular exponentiations per vote: $H^{sk_i} \bmod N^2$ and $(1 + N)^m \bmod N^2$. With 255-bit N , each exponentiation operates on 510-bit numbers. On a single core, this yields approximately 150–200 votes/second. With 16 parallel workers, the aggregate throughput reaches 2,000–3,000 votes/second.

The bottleneck is not the encryption itself but the bcrypt hashing for voter passwords. By pre-computing a single hash and reusing it, the test eliminates this overhead.

B. Database Insertion

PostgreSQL COPY achieves sustained write rates of 50,000+ rows/second. Each row contains approximately 400 bytes (two 127-character hex strings for c_i and aux_i , plus voter metadata). For 30 million voters, this produces approximately 12 GB of data.

C. Aggregation Latency

The aggregation phase consists of:

- **Auxiliary product computation:** n modular multiplications mod N^2 . For $n = 5,000$, this takes < 0.01 s.
- **Ciphertext product:** n modular multiplications mod N^2 (in libtommath via `mp_mod_mul`).
- **Decryption:** One `mp_exptmod` (product^{sk_A}), one `mp_invmod` (aux^{-1}), one `mp_mod_mul`, one division, and one final `mp_mod_mul`.

The decryption step is constant-time regardless of voter count, as it operates on fixed-size 510-bit numbers. The product computation scales linearly with n . For $n = 5,000$, total aggregation time is 0.1 seconds.

D. Comparison with JS BigInt

We initially attempted to use JavaScript’s native `BigInt` for all aggregation math. While correct, the native `libtom-math` C module provides approximately $3\text{--}5\times$ faster modular exponentiation due to optimized internal representations and assembly-level optimizations in `libtommath`. The tradeoff is the added complexity of maintaining a native addon with correct byte ordering.

XIII. RELATED WORK

a) Helios: [2] is a web-based open-audit voting system using homomorphic encryption. It relies on a trusted authority for decryption and does not use blockchain for immutability.

b) Civitas: [3] extends Helios with coercion resistance using distributed plaintext equivalence tests and enforced eligibility. NARAD does not address coercion resistance but could benefit from similar techniques.

c) Aggregator-Oblivious Encryption: The protocol in [4] describes a scheme where users encrypt without key coordination, using auxiliary values to mask the aggregator’s view. NARAD implements this protocol end-to-end with concrete engineering choices (WASM, `libtommath`, Solana).

d) Blockchain Voting: Systems like Follow My Vote and Voatz use blockchain for vote recording but typically do not employ homomorphic encryption for aggregation. NARAD combines both: on-chain immutability for individual vote records and off-chain homomorphic aggregation for privacy-preserving tallying.

XIV. DEPLOYMENT CONSIDERATIONS

A. From Localnet to Mainnet

The current deployment uses a Solana test validator, which provides no real-world security. Moving to Solana mainnet requires:

- **Program audit:** The Anchor program should undergo formal verification and third-party security audit before mainnet deployment.
- **Wallet security:** The backend’s Solana wallet (currently auto-generated) should be replaced with a hardware wallet or multi-sig (e.g., Squads Protocol) to prevent single-point-of-failure key compromise.
- **Account limits:** Solana mainnet has account size limits and rent costs. The `ElectionData` account size scales with `total_candidates`: $\text{space} = 8 + 1 + 32 + 8 + 8 + 4 + k \times 36 + 660$ bytes. For 10 candidates, this is $\sim 1,078$ bytes, costing ~ 0.008 SOL in rent.
- **Transaction throughput:** Solana mainnet supports $\sim 65,000$ TPS, but vote transactions involve account creation (PDA) which is more expensive. Realistic throughput for on-chain vote recording is $\sim 1,000$ votes/second.

B. Scaling the Paillier Modulus

For elections requiring more than 10 candidates, a larger N is needed. Table II shows the relationship between candidate count, required N size, and example prime sizes.

TABLE II
PAILLIER MODULUS SCALING FOR CANDIDATE COUNT

Candidates	Bits needed	N bits	Prime size
10	250	255	128-bit
20	500	505	253-bit
50	1,250	1,255	628-bit
100	2,500	2,505	1,253-bit

Larger N values increase encryption and aggregation time proportionally, as modular exponentiation complexity scales with bit length. A 2,505-bit N would be roughly $10\times$ slower than the current 255-bit N .

C. High Availability

The current single-container deployment is suitable for testing but not for production. A production deployment would need:

- **Backend replication:** Multiple backend instances behind a load balancer (e.g., `nginx` or `HAProxy`), with shared PostgreSQL (primary-replica) and Solana RPC endpoint.
- **Database replication:** PostgreSQL streaming replication with automatic failover (e.g., using `Patroni`).
- **CDN for WASM:** The `encryption.wasm` and `encryption.js` files should be served via a CDN for low latency.
- **Monitoring:** The admin panel’s health check dashboard should be backed by Prometheus/Grafana for real-time alerting.

D. CI/CD Pipeline

A GitHub Actions workflow (`.github/workflows/ci.yml`) is included for continuous integration:

- **Lint stage:** TypeScript type checking and Prettier formatting.
- **Test stage:** Runs backend tests against a PostgreSQL service container.
- **WASM build:** Builds the WebAssembly module in an Emscripten container.
- **Docker build:** Builds and pushes Docker images to GitHub Container Registry.

The CI pipeline ensures that every pull request is type-checked, tested, and buildable before merge.

XV. CONCLUSION

We presented NARAD, a complete implementation of a Paillier-based aggregator-oblivious voting system deployed as containerized microservices on Solana. The system achieves end-to-end vote privacy through client-side WASM encryption and privacy-preserving aggregation through homomorphic properties. We documented seven critical implementation bugs—ranging from an invalid even Paillier modulus to byte-order mismatches in native bindings—and provided formal correctness proofs for the aggregation pipeline under stated assumptions.

The system supports up to 33.5 million voters across 10 candidates with sub-second aggregation, limited by the bit-width of the Paillier modulus. Future work includes scaling to larger elections via bigger primes, integrating zero-knowledge proofs for individual vote validity, and deploying on Solana mainnet with multi-validator replication.

REFERENCES

REFERENCES

- [1] P. Paillier, “Public-key cryptosystems based on composite degree residuosity classes,” in *Proc. EUROCRYPT*, 1999, pp. 223–238.
- [2] B. Adida, “Helios: Web-based open-audit voting,” in *Proc. USENIX Security Symposium*, 2008, pp. 335–348.
- [3] A. Kiayias, M. Korman, and D. Walluck, “Voting without self-enforcing protocols: A survey of e-voting schemes,” in *Proc. ACNS*, 2008.
- [4] “Aggregator-oblivious encryption,” *IACR Cryptology ePrint Archive*, 2014. [Online]. Available: <https://eprint.iacr.org/2014/>
- [5] “Privacy-preserving aggregation protocols,” *Proceedings on Privacy Enhancing Technologies (PoPETS)*, vol. 2023, no. 1, 2023.
- [6] “libtommath: A portable number theoretic multiple-precision integer library,” [Online]. Available: <https://github.com/libtom/libtommath>
- [7] “Emscripten: LLVM to WebAssembly compiler,” [Online]. Available: <https://emscripten.org>
- [8] “Anchor: Solana Sealevel Framework,” [Online]. Available: <https://github.com/coral-xyz/anchor>
- [9] “Solana: A fast, secure, and censorship-resistant blockchain,” [Online]. Available: <https://solana.com>
- [10] “Docker: Containerized application platform,” [Online]. Available: <https://docker.com>
- [11] “Prisma: Next-generation Node.js and TypeScript ORM,” [Online]. Available: <https://prisma.io>
- [12] “React: A JavaScript library for building user interfaces,” [Online]. Available: <https://react.dev>
- [13] “Tailwind CSS: A utility-first CSS framework,” [Online]. Available: <https://tailwindcss.com>
- [14] “Docker Compose: Multi-container Docker applications,” [Online]. Available: <https://docs.docker.com/compose/>
- [15] “Node-API: Node.js native addon API,” [Online]. Available: <https://nodejs.org/api/n-api.html>